Assessment 1 Lab Report

Foundations of Robotics

Ruhan Shafi

UNIVERSITY OF CANBERRA

Assessment 1: Lab Report

Lab Group: Group 8 - Ruhan Shafi (u3284342) and Alex Scola - (u3324662)

1 Abstract

This report presents the design, implementation, and evaluation of a light-reactive autonomous robot control algorithm for the Sphero RVR mobile robot platform. The objective of the assignment was to develop a Sense-Think-Act (STA) control loop using ROS 2 and the `rclpy` Python library, such that the robot detects ambient light intensity above a defined threshold and responds by spinning in place, indefinately.

The implemented system subscribes to the RVR's front-mounted ambient light sensor topic, continuously comparing the measured illuminance (in Lux) against a calibrated threshold of 750 Lux. Upon detection of a bright light stimulus, the algorithm publishes angular velocity commands to the robot's motor controller via the `/cmd_vel` topic, causing the RVR to spin at 1.5 rad/s. A timer-based control loop operating at 10 Hz ensured responsive and consistent actuation.

2 Introduction

2.1 Background and Context

Autonomous mobile robots must continuously interact with their physical environment to perform useful tasks. At the foundation of most autonomous behaviour is a fundamental control paradigm known as the Sense-Think-Act (STA) loop. First formalised in behaviour-based robotics literature, the STA model describes a repeating cycle in which a robot reads data from its sensors (`Sense`), applies logic to decide on a course of action (`Think`), and then issues commands to its actuators (`Act`). This simple but powerful abstraction underpins everything from industrial manipulators to autonomous vehicles.

The Sphero RVR is a differential-drive mobile robot platform equipped with a suite of onboard sensors including an ambient light sensor, a colour sensor, an IMU, and infrared emitters. It runs a ROS 2 application stack on an onboard Raspberry Pi, exposing sensor data and accepting motor commands through standard ROS 2 topics.

2.2 Learning Objectives

- Understand and apply the Sense-Think-Act control paradigm in a physical robotics context.
- Subscribe to a ROS 2 sensor topic and correctly interpret the received message data.
- Publish velocity commands to a ROS 2 actuator topic to control robot movement.
- Design and implement a timer-based control loop using `rclpy` that operates at a defined frequency.
- Calibrate a sensor threshold empirically and reason about its effect on system behaviour.
- Handle edge cases such as timed actuation windows and safe robot shutdown.

3 Materials and Methods

3.1 Equipement and Software

Table 1 lists all hardware and software components used in this experiment.

Component	Details
Sphero RVR	Differential-drive rover with onboard sensors and motor controllers
Onboard Raspberry Pi	Runs the ROS 2 application stack and FastDDS discovery server
Ambient Light Sensor	Front-mounted digital sensor; publishes illuminance in Lux
Laptop / Jupyter Container	Runs `rclpy` nodes and the STA algorithm over Wi-Fi

Table 1: Hardware and software components used in the experiment

3.2 Network and Communication Setup

The RVR and the laptop operated on the same Wi-Fi subnet. Because multiple groups shared the network simultaneously, a FastDDS Discovery Server running on the RVR's Raspberry Pi was used to isolate and manage node discovery. Each group was assigned a unique `ROS_DOMAIN_ID` to prevent cross-group topic interference with `ROS_DOMAIN_ID` being `8`.

The following environment variables were configured on the laptop before initialising `rclpy`:

```
CONSOLE_HOSTNAME = "192.168.1.208"   #laptop IP/hostname

DISCOVERY_SERVER_PORT = 11811
ROS_DOMAIN_ID = "8"

# ROS 2 networking (Fast DDS discovery server)
os.environ["ROS_DOMAIN_ID"] = ROS_DOMAIN_ID
os.environ["ROS_HOSTNAME"] = CONSOLE_HOSTNAME
os.environ["RMW_IMPLEMENTATION"] = "rmw_fastrtps_cpp"
os.environ["ROS_AUTOMATIC_DISCOVERY_RANGE"] = "SUBNET"
os.environ["ROS_DISCOVERY_SERVER"] = f"{DISCOVERY_SERVER_HOST}:{DISCOVERY_SERVER_PORT}"
os.environ["ROS_SUPER_CLIENT"] = "True"
```

These variables were set programmatically in Python using `os.environ` before any `rclpy` calls, ensuring the middleware was correctly configured at the point of initialisation. `rclpy` was then initialised once per kernel session with a guard to prevent double-initialisation, using the following code

```
if not rclpy.ok():
  rclpy.init()
  print("rclpy initialised")
else:
  print("rclpy already running")
```

3.3 ROS 2 Topics Used - Software Stack

ROS 2 Topic	Message Type	Direction	Purpose
/ambient_light_broadcaster /ambient_light	sensor_msgs/Illuminance	Subscribe	Ambient lux reading from RVR sensor
/cmd_vel	geometry_msgs/Twist	Publish	Linear and angular velocity commands to motors

Table 2: ROS 2 topics used for sensing, actuation and timing

The `Illuminance` message received on `/ambient_light_broadcaster/ambient_light` contains a single floating-point field, `illuminance`, representing the measured light intensity in Lux. The `TwistStamp` message contains linear and angular velocity vectors; this implementation used only `twist.linear.x` (forward speed) and `twist.angular.z` (yaw rate), both in SI units.

3.4 Threshold Calibration Procedure

Before implementing the full STA algorithm, the ambient light sensor was observed independently using a dedicated subscriber node in a debug cell (as seen below)to verify that the sensor was working properly and was able to transmit it's reading.

```
from rclpy.node import Node
from sensor_msgs.msg import Illuminance


class LightSensorReaderNode(Node):
  """Subscribes to the illuminance topic and prints each reading."""

  def __init__(self):
    super().__init__('light_sensor_reader_node')
      self.light_sub = self.create_subscription(
      Illuminance,
```

```
      '/ambient_light_broadcaster/ambient_light',
      self.light_callback,
      10
    )
    self.get_logger().info('LightSensorReaderNode started - listening for illuminance...')

  def light_callback(self, msg: Illuminance):
    """Called every time a new illuminance message arrives."""
    lux = msg.illuminance
    print(f'Illuminance: {lux:.2f} lux')


def main():
  print('Starting LightSensorReaderNode...')
  try:
    node = LightSensorReaderNode()
    rclpy.spin(node)          # blocks; prints readings as they arrive
  except KeyboardInterrupt:
    print('Interrupted')
  finally:
    print('Node shutdown.')


print('LightSensorReaderNode declared - call main() to run it')
```

During this debuging, Lux readings were recorded under two conditions to establish a suitable threshold:

- Ambient lighting of the laboratory
- Phone torch held approximately 10 cm from the sensor

Readings were observed in real time from the printed console output of the `LightSensorReaderNode`. The threshold was then set to a value clearly above the ambient baseline but reliably triggered by the phone torch at approximately 10 cm distance. An initial value of 750 Lux was chosen.

3.5 Algorithm Design and Implementation

The STA logic was implemented as a single `rclpy` Node class named `SenseThinkActNode`. The design separates the three phases clearly using ROS 2 commands.

3.5.1 Sense Phase

A subscriber to `/ambient_light_broadcaster/ambient_light` triggers the `_sense_cb` callback each time a new light reading arrives (every ROS clock cycle). The callback stores the latest lux value in the instance variable `self.current_lux`, making it available to the control loop.

```
class SenseThinkActNode(Node):
  def __init__(self):
    super().__init__('sense_think_act_node')

    # ---- SENSE: light sensor subscriber ---- #
    self.light_sub = self.create_subscription(
      Illuminance,
      '/ambient_light_broadcaster/ambient_light',
      self._sense_cb,
      10
    )

    self.spin_end_time: float = 0.0

  def _sense_cb(self, msg: Illuminance):
    self.current_lux = msg.illuminance
    self.get_logger().debug(f'Sensed: {self.current_lux:.1f} lux')
```

3.5.2 Think and Act Phase

A ROS 2 timer fires at 10 Hz, triggering the `_think_and_act_cb` callback. This callback implements both the Think and Act phases in sequence. In the Think phase, the current lux value is compared against the `LIGHT_THRESHOLD`

constant. If the light is bright (i.e greater than the constant), the `spin_end_time` is updated to the current time plus five seconds, resetting the spin window. The `should_spin` flag is then derived from whether the current time still falls within the spin window. In the Act phase, the appropriate `TwistStamped` command is published based on `should_spin` depending on current time is greater than less than `spin_end_time`.

The choice was made to use the ROS clock (`get_clock().now()`) rather than Python's `time.time()` to avoid any ROS time overrides. The `spin_end_time` approach seperates the actuation duration from the instantaneous sensor state. This means that the robot continues spinning for the full five seconds even if the light is removed immediately after being detected, which will happen unless the light moves with the RVR.

```python
class SenseThinkActNode(Node):
    def __init__(self):
    super().__init__('sense_think_act_node')

  # ---- ACT: velocity command publisher ---- #
  self.cmd_vel_pub = self.create_publisher(TwistStamped, '/cmd_vel', 10)

  # ---- THINK: timer drives the control loop ---- #
  self.timer = self.create_timer(
    1.0 / self.LOOP_RATE_HZ,
    self._think_and_act_cb
  )

  def _sense_cb(self, msg: Illuminance):
    self.current_lux = msg.illuminance
    self.get_logger().debug(f'Sensed: {self.current_lux:.1f} lux')

  def _think_and_act_cb(self):
    cmd = TwistStamped()

    # Get the current ROS time in seconds
    now = self.get_clock().now().nanoseconds / 1e9

    # ---- THINK ---- #
    light_is_bright = self.current_lux > self.LIGHT_THRESHOLD

    if light_is_bright:
      # Light detected - (re)start the 5-second spin window
      self.spin_end_time = now + 5.0
        self.get_logger().info(
        f'BRIGHT ({self.current_lux:.1f} lux > {self.LIGHT_THRESHOLD}) '
        f'-> spin window reset, stopping at t={self.spin_end_time:.1f}'
        )

    should_spin = now < self.spin_end_time

    if should_spin:
      # ---- ACT: spin ---- #
      cmd.twist.linear.x  = 0.0
      cmd.twist.angular.z = self.ANGULAR_SPEED
      self.get_logger().info(
      f'SPINNING - {self.spin_end_time - now:.1f}s remaining'
      )
    else:
      # ---- ACT: stop ---- #
      cmd.twist.linear.x  = 0.0
      cmd.twist.angular.z = 0.0
      self.get_logger().info(
      f'DIM ({self.current_lux:.1f} lux) -> STOPPED'
      )

    self.cmd_vel_pub.publish(cmd)

```

3.5.3 Safe Shutdown

A `stop_robot()` method publishes a zero-velocity `TwistStamp` command and is called unconditionally in the `finally` block of the main function. This guarantees that the robot halts safely regardless of how the node is stopped, including keyboard interrupts and kernel restarts.

```python
def stop_robot(self):
  """Publish a zero-velocity command to safely halt the robot."""
  stop_cmd = TwistStamped()
  stop_cmd.twist.linear.x  = 0.0
  stop_cmd.twist.angular.z = 0.0
  self.cmd_vel_pub.publish(stop_cmd)
  self.get_logger().info('Safety stop sent')
```

3.6 Experimental Procedure

The experiment was conducted as follows:

1. ROS environment variables and drivers were configured and `rclpy` was initialised.
2. The `LightSensorReaderNode` was run to record baseline and stimulus lux values.
3. The `LIGHT_THRESHOLD` was set based on the observed readings.
4. The `SpinTestNode` was run to verify motor control independently.
5. The full `SenseThinkActNode` was launched.
6. A phone torch was directed at the sensor from varying distances. The robot behaviour and console log output were observed and recorded.
7. The threshold was adjusted and the test repeated until the behaviour matched the expected outcome consistently.
8. The kernel was restarted to confirm safe shutdown and stop behaviour.

4 Results and Discussion

4.1 Robot Behavioural Results

With the threshold set to 750 Lux and the `SenseThinkActNode` running, the following behaviours were consistently observed:

- When the phone torch was directed at the sensor, the robot began spinning.
- The robot continued spinning for the full five-second window after each light stimulus, regardless of whether the torch was held in place or immediately removed.
- After the spin window elapsed, the robot stopped and remained stationary until a new stimulus was detected.
- Repeated stimuli applied during an active spin window correctly reset the `spin_end_time`, extending the spin rather than restarting it from zero.
- On kernel restart, the `stop_robot()` safety call successfully halted the motors in all tested cases.

4.2 Analysis - STA Loop Performance

The results confirm that the three-phase STA architecture performed as intended. The *Sense* phase: a lightweight subscriber callback that only stores the latest lux value. The *Think* phase, implemented inside the 10 Hz timer callback, correctly derived the `should_spin` decision from the stored sensor state on every tick. The *Act* phase published the appropriate `TwistStamp` command consistently and without error.

4.3 Potential Improvements

Several improvements could be made to increase the robustness and capability of the system:

- **Hysteresis band.** Rather than a single threshold, a lower *deactivation* threshold could be introduced so that the robot only stops spinning once the lux value falls below, for example, 450 Lux, preventing rapid on-off switching near the threshold boundary.

- **Proportional response.** Rather than binary spin-or-stop behaviour, the angular velocity could be scaled proportionally to the measured lux value
- **Multi-sensor fusion.** Adding a second sensor on the rear or side of the robot would allow direction-of-stimulus estimation, enabling the robot to turn *towards* the light source rather than spinning arbitrarily.

5 Conclusion

In conclusion, the assignment achieved its primary goal of implementing a working STA loop on a physical robot, and provided the group with valuable practical experience with ROS 2 publisher-subscriber communication, timer-based control, and the real-world constraints of network robotics. The skills learnt from this assessment such as ROS node architecture are directly transferable to more complex autonomous systems that will be present in future lab assessments.